Lab 12 — Praktijklab: uitrollen, bijwerken, terugkeren

Doel: de API uitrollen in twee replica's met sondes, ze opschalen, ze zonder onderbreking naar versie 1.1 brengen, een kapotte versie proberen en de rollout zien blokkeren, terugkeren, en dan bewust een readiness en een liveness breken om hun tegengestelde effecten te zien.

Vereisten — Lab 11 afgewerkt: k3s geïnstalleerd, `localhost/api:1.0` geïmporteerd in het cluster, Podman met `api:1.0`.

Meegeleverde bestanden (`files/`) - `deployment-api.yaml` — het Deployment van de API: twee replica's, readiness en liveness op `/actuator/health`. - `Dockerfile.v11` — bouwt `api:1.1` (een ander standaardbericht). - `Dockerfile.bad` — bouwt `api:bad` (een image die niet start).

Stap 0 — Twee images extra in het cluster

```
mkdir -p ~/labo-docker/12 && cd ~/labo-docker/12 && cp <pad-naar-het-lab>/files/* .
podman build -q -f Dockerfile.v11 -t api:1.1 . && podman build -q -f Dockerfile.bad -t api:bad
.
podman save localhost/api:1.1 | sudo k3s ctr images import -
podman save localhost/api:bad | sudo k3s ctr images import -
sudo k3s crictl images | grep 'localhost/api'
```

Observeer twee builds uit de cache (enkele seconden), twee imports, en drie regels `localhost/api` in het cluster: `1.0`, `1.1`, `bad`.

Stap 1 — Het Deployment

```
kubectl create namespace lab12 ; kubectl config set-context --current --namespace=lab12
kubectl apply -f deployment-api.yaml
kubectl rollout status deployment/api --timeout=60s
kubectl get deployment,replicaset,pods -o wide
```

Observeer `Waiting for deployment "api" rollout to finish: 1 of 2 updated replicas are available...` en dan `successfully rolled out`; drie niveaus van objecten: `deployment.apps/api 2/2`, `replicaset.apps/api-6c7bb54fc5 2 2 2`, en twee Pods `api-6c7bb54fc5-xxxxx Running` elk met hun IP.

```
kubectl get pods --show-labels
kubectl get pods -l app=api -o name
kubectl describe deployment api | grep -E 'Replicas:|StrategyType|RollingUpdateStrategy'
```

Observeer de labels `app=api,pod-template-hash=6c7bb54fc5` op elke Pod, de twee Pods gefilterd met `-l app=api`, en de strategie: `RollingUpdate`, `25% max unavailable, 25% max surge`.

Uitleg. Jij schreef een Deployment; het cluster maakte een ReplicaSet (genoemd naar de hash van het template) en twee Pods, verbonden door de labels. De `rollout status` wachtte tot de readiness probes slaagden.

Stap 2 — Opschalen

```
kubectl scale deployment api --replicas=3
kubectl rollout status deployment/api --timeout=60s
kubectl get pods ; kubectl get rs
```

Observeer een derde Pod met **dezelfde** hash, en de ReplicaSet op `3 3 3`.

Uitleg. `scale` verandert `spec.replicas` ; de ReplicaSet doet de rest. Geen nieuwe ReplicaSet: het template is niet veranderd.

Stap 3 — Naar 1.1 zonder onderbreking

```
kubectl set image deployment/api api=localhost/api:1.1
kubectl rollout status deployment/api --timeout=90s
kubectl get rs
kubectl rollout history deployment/api
kubectl exec $(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}') -- wget -
qO- http://localhost:8080/ ; echo
kubectl get events --sort-by=.lastTimestamp | grep ScalingReplicaSet | tail -6 | cut -c1-110
```

Observeer het verloop `1 out of 3 new replicas have been updated…` , `2 out of 3…` , `1 old replicas are pending termination…` , `successfully rolled out` ; twee ReplicaSets, de nieuwe op `3 3 3` en de oude op `0 0 0` ; twee revisies in de geschiedenis; `{"message":"version 1.1",…}` ; en de dans van de Events: nieuwe `0 → 1` , `1 → 2` / oude `3 → 2` , oude `2 → 1` / nieuwe `2 → 3` , oude `1 → 0` .

Uitleg. Nooit minder dan drie klare Pods: een nieuwe wordt aangemaakt, klaar afgewacht, dan een oude weggehaald. De oude ReplicaSet blijft op nul: dat is revisie 1, klaar voor een terugkeer.

Stap 4 — Een kapotte versie

```
kubectl set image deployment/api api=localhost/api:bad
sleep 25 ; kubectl get pods
kubectl rollout status deployment/api --timeout=20s ; echo "exit=$?"
kubectl get rs
kubectl logs $(kubectl get rs --sort-by=.metadata.creationTimestamp -o name | tail -1) --tail=1
```

Observeer drie Pods `1/1 Running` van de 1.1 en **één enkele** nieuwe Pod `0/1 Error` met `RESTARTS 2` ; een `rollout status` die wacht (`1 out of 3 new replicas have been updated…`) en dan `error: timed out waiting for the condition` , `exit=1` ; drie ReplicaSets waarvan de nieuwe op `1 1 0` ; en de oorzaak, gelezen in de logs van de nieuwe ReplicaSet: `demarrage impossible : configuration manquante` .

```
kubectl rollout undo deployment/api
kubectl rollout status deployment/api --timeout=60s
kubectl get pods ; kubectl rollout history deployment/api
kubectl get deployment api -o jsonpath='{.spec.template.spec.containers[0].image}' ; echo
```

Observeer `deployment.apps/api rolled back`, de kapotte Pod in `Terminating` (of nog `Error` tot hij verdwijnt), de drie Pods van de 1.1 onaangeroerd, een geschiedenis `1, 3, 4` (revisie 2 werd 4 door de terugkeer), en de image terug op `localhost/api:1.1`.

Uitleg. Het Deployment heeft geen enkele gezonde Pod weggehaald: `maxUnavailable` van 25 % op drie replica's is nul. De dienst is tijdens het incident niet geraakt, en de terugkeer kostte één commando.

Stap 5 — Een readiness die faalt

```
kubectl patch deployment api --type=json -
p='[{"op":"replace","path":"/spec/template/spec/containers/0/readinessProbe/httpGet/port","valu
e":8081}]'
sleep 25 ; kubectl get pods
kubectl describe pods -l app=api | grep 'Readiness probe failed' | tail -1 | cut -c1-120
kubectl get deployment api
kubectl rollout status deployment/api --timeout=10s ; echo "exit=$?"
kubectl rollout undo deployment/api ; kubectl rollout status deployment/api --timeout=60s
```

Observeer één nieuwe Pod `0/1 Running` (hij draait, maar niets luistert op `8081`) naast de drie oude `1/1`; `Readiness probe failed: Get "http://10.42.0.x:8081/actuator/health": dial tcp … connection refused`; `READY 3/3 UP-TO-DATE 1 AVAILABLE 3`; een `rollout status` die verloopt; en dan de terugkeer.

Uitleg. Een Pod die nooit klaar is, krijgt geen verkeer en laat niet toe een oude weg te halen: de rollout blokkeert zonder schade. Dat is het gedrag dat je wilt wanneer een nieuwe versie niet correct start.

Stap 6 — Een liveness die faalt

```
kubectl patch deployment api --type=json -
p='[{"op":"replace","path":"/spec/template/spec/containers/0/livenessProbe/httpGet/port","value
":8081},
{"op":"replace","path":"/spec/template/spec/containers/0/livenessProbe/initialDelaySeconds","va
lue":2},
{"op":"replace","path":"/spec/template/spec/containers/0/livenessProbe/periodSeconds","value":3
}]'
sleep 40 ; kubectl get pods
kubectl describe pods -l app=api | grep -E 'Liveness probe failed|Killing' | tail -2 | cut -c1-
120
```

Observeer deze keer **drie** nieuwe Pods, allemaal `0/1`, in `CrashLoopBackOff` (of `Running` tussen twee herstarts) met `RESTARTS 2`: de rollout heeft de oude vervangen (de readiness slaagde), en

dan heeft de liveness elke nieuwe Pod gedood; `Liveness probe failed: … connection refused` en `Container api failed liveness probe, will be restarted`.

```
kubectl rollout undo deployment/api ; kubectl rollout status deployment/api --timeout=60s
kubectl get pods ; kubectl rollout history deployment/api
```

Observeer de drie gezonde Pods terug, de kapotte in `Terminating`, en een geschiedenis die opnieuw nummers overslaat.

Uitleg. De rollout kijkt alleen naar de readiness; een gebroken liveness werkt achteraf, op Pods die al als goed meegeteld zijn, en neemt de hele dienst mee. Readiness: beschermt de uitrol; liveness: met zorg in te stellen.

Opruimen

```
kubectl delete namespace lab12
kubectl config set-context --current --namespace=default
sudo k3s crictl rmi localhost/api:bad
podman rmi api:bad
```

Observeer de namespace verwijderd met zijn objecten, en de kapotte image weggehaald uit het cluster en uit Podman. Hou `api:1.0` en `api:1.1` aan beide kanten: lab 13 rolt de volledige keten uit.

Wat je nu moet kunnen beweren

- Een `apply` van een Deployment maakt een ReplicaSet `api-<hash>` en zijn Pods aan, verbonden door `app=api` en `pod-template-hash`.
- `scale` behoudt de ReplicaSet; `set image` maakt een nieuwe aan en verplaatst de replica's een voor een: `0 → 1`, `3 → 2`, … nooit minder dan drie klaar.
- Een kapotte image = één enkele Pod `Error`, drie oude onaangeroerd, `rollout status` in `timed out`, `logs rs/…` voor de oorzaak, `rollout undo` om te herstellen.
- Gebroken readiness = `0/1 Running`, rollout geblokkeerd, dienst intact; gebroken liveness = alles in `CrashLoopBackOff`.
- `rollout history` slaat nummers over na elke `undo`: normaal.